AUTOMATED SECURITY ASSESSMENT & REPORTING PIPELINE

รายงานการแก้ไขระบบ ขั้นที่ 3

เก็บงานขึ้น git · ทดสอบอัตโนมัติ · ล้างข้อมูลผิด · backup · CI
ต่อจาก "รายงานการแก้ไขระบบ ขั้นที่ 1–2"

> โปรเจกต์: `C:\Allprojects\Projects`
> GitHub: `github.com/jjjecop/security-pipe-ine` (private)
> ช่วงเวลา: 6–8 ตุลาคม 2026

1. สรุปสั้นๆ

ขั้นที่ 1–2 ทำให้ระบบ**ทำงานถูก** ส่วนขั้นที่ 3 ทำให้ระบบ**ไม่พังง่ายและกู้คืนได้** คือทุกการแก้ไขมีประวัติย้อนกลับได้ มี test คอยจับว่าโค้ดพังไหม มี backup ของข้อมูล และมีเอกสารบอกวิธีติดตั้งและใช้งาน

เรื่อง	ก่อน	หลัง
ประวัติการแก้โค้ด	มีแค่ commit Phase 0 งานทั้งหมดอยู่ในเครื่องเครื่องเดียว	16 commit แยกตามเรื่อง สำรองบน GitHub
ติดตั้งบนเครื่องใหม่	ต้องเดาว่าใช้ library อะไร เวอร์ชันไหน	`pip install -r requirements-dev.txt` คำสั่งเดียว
รู้ได้ยังไงว่าแก้แล้วพัง	ต้องรันสแกนจริง แล้วดูรายงานเอง	`pytest` 17 test เสร็จในไม่กี่วินาที
ลืมรัน test	เป็นไปได้	GitHub Actions รันให้ทุกครั้งที่ push และขึ้น ☑ หรือ ❌
ข้อมูลในฐานข้อมูล	มี `fixed` ปลอม 358 รายการ	ลบแล้ว ตัวเลขตรงกับความจริง
ฐานข้อมูลหาย	กู้ไม่ได้	มีไฟล์ backup จาก `pg_dump` ทดสอบแล้วว่าเปิดได้
`verify_findings.py`	ผูกกับ NodeGoat ด้วย path ตายตัว	ใช้ `--project` กับโปรเจกต์ไหนก็ได้
เอกสาร	ไม่มี	`README.md` มีวิธีติดตั้ง ใช้งาน และแก้ปัญหา
ไฟล์ขยะ	template รายงานเก่า PDF เก่า และผล ZAP เก่าวางอยู่ปนกัน	ลบแล้ว และกันไม่ให้ไฟล์ผลลัพธ์ขึ้น git อีก

ลำดับงานที่ทำ

ข้อ	งาน	หัวข้อในเล่มนี้
1	ตั้งค่า certificate ของ Norton	2
2	ขึ้น git และ GitHub	3
3	`requirements.txt`	4
4	pytest	5
5	กลุ่ม A: เก็บกวาดระบบ (ลบไฟล์เก่า, ลบ finding ซ้ำ, backup, verify_findings, README)	6
6	กลุ่ม B: GitHub Actions	7

2. ตั้งค่า certificate ของ Norton

1 Semgrep ต่อ internet ไม่ได้เพราะ Norton ทำให้ SAST ใช้ไม่ได้

ปัญหาเดิม

Semgrep ขึ้น `SSLError: CERTIFICATE_VERIFY_FAILED` ตอนโหลดกฎจาก semgrep.dev ทำให้ ScanJob 29 ได้สถานะ `partial` (สแกนอื่นผ่าน แต่ SAST ล้ม)

สาเหตุ

Norton Antivirus ดักการเชื่อมต่อ HTTPS เพื่อสแกนหาไวรัส โดยใช้ certificate ของตัวเองชื่อ Norton Web/Mail Shield Root เบราว์เซอร์รู้จัก cert นี้เพราะ Windows เชื่อถือ แต่ Python ใช้รายการ cert ของตัวเอง (`certifi`) ซึ่งไม่มี cert ของ Norton เลยมองว่าเป็นการเชื่อมต่อปลอม

แก้อย่างไร

สร้างไฟล์ `ca-bundle.pem` = cert ทั้งหมดของ certifi + cert ของ Norton (`C:\ProgramData\Norton\Antivirus\wscert.pem`) แล้วบอก Python ให้ใช้ไฟล์นี้ผ่าน `.env` :

```
REQUESTS_CA_BUNDLE=C:\Allprojects\Projects\ca-bundle.pem
SSL_CERT_FILE=C:\Allprojects\Projects\ca-bundle.pem
```

ทำไมไม่ปิดการตรวจ SSL ไปเลย

ทำได้แต่อันตราย เพราะจะเชื่อทุกการเชื่อมต่อรวมถึงของคนร้ายด้วย ซึ่งไม่ควรมีในเครื่องมือด้าน security วิธีนี้เพิ่มแค่ cert ของ Norton ตัวเดียว

ได้อะไร

ScanJob 32 ได้สถานะ `completed` และ Semgrep เจอ 35 จุด

ไฟล์: `ca-bundle.pem` (ใหม่ อยู่ใน `.gitignore` เพราะใช้ได้แค่เครื่องนี้), `.env`

3. ขึ้น git และ GitHub

git คือระบบบันทึกประวัติโค้ด แต่ละ **commit** เป็นเหมือนจุด save ในเกม ถ้าแก้แล้วพังก็ย้อนกลับได้ ส่วน **GitHub** คือที่เก็บสำรองบน internet ถ้าเครื่องเสียโค้ดก็ยังอยู่

2 `.gitignore` : กันไฟล์ที่ไม่ควรขึ้น git ความปลอดภัย

สิ่งที่ ignore	เหตุผล
`.env`	มีรหัสผ่าน Postgres และ `GEMINI_API_KEY` ถ้าหลุดขึ้น GitHub คนอื่นเอาไปใช้ได้ และลบออกจากประวัติ git ยากมาก
`venv/` , `__pycache__/`	สร้างใหม่ได้จาก `requirements.txt` และไฟล์ใหญ่
`reports/` , `security_report.pdf` , `zap_results.json`	เป็นผลลัพธ์ที่สร้างใหม่ทุกรอบ และมีรายละเอียดช่องโหว่ของระบบ
`ca-bundle.pem`	ใช้ได้เฉพาะเครื่องที่มี Norton
`CheatSheetSeries/`	repo ของ OWASP ขนาด 2.5 GB และเป็น git repo ซ้อนอยู่ข้างใน clone ใหม่ได้เสมอ

`backups/`	มีข้อมูลช่องโหว่ทั้งหมดในฐานข้อมูล ต้องเก็บเป็นความลับ

3 แยก commit ตามเรื่อง

ทำอย่างไร

แทนที่จะ commit ทุกอย่างทีเดียว เราแยกเป็นเรื่องๆ เช่น models, scanners, Gemini, report, tests เปลี่ยนชื่อ branch เป็น `main` และ push ขึ้น GitHub

ได้อะไร

อ่านประวัติแล้วรู้ว่าแต่ละช่วงทำอะไรไป ถ้าเรื่องไหนมีปัญหาก็ย้อนเฉพาะเรื่องนั้นได้

ประวัติ commit ทั้งหมด

commit	วันที่	ข้อความ
`c2a8480`	2026-09-01	Phase 0: initial project setup with models, docker-compose, and alembic
`e19a1c7`	2026-10-06	Ignore secrets, reports and CheatSheetSeries
`906d675`	2026-10-06	Add database models, connection and alembic migrations
`f997bb4`	2026-10-06	Add scanners with error handling, normalizer and finding dedup
`1d82209`	2026-10-06	Add Gemini remediation from OWASP Cheat Sheets and MITRE CWE
`44cd32f`	2026-10-06	Add report generation and single-command run_pipeline
`e614aea`	2026-10-06	Add tool exploration scripts under tests
`909aa79`	2026-10-06	Add requirements.txt
`33e8f5b`	2026-10-06	Use ASCII comment in requirements.txt
`b12b9cf`	2026-10-08	Add CWE to OWASP mapping data and seed script (มีการย้าย tests → scripts ติดมาด้วย ดูหัวข้อ 8)
`79c8f4e`	2026-10-08	Add pytest tests for risk scoring and fingerprints
`417a118`	2026-10-08	Remove old report templates replaced by reports/ folder
`b968467`	2026-10-08	Make verify_findings.py take --project and --lockfile arguments
`059fbcf`	2026-10-08	Add README with setup and usage guide
`4d80cf1`	2026-10-08	Ignore database backups folder
`ea24308`	2026-10-08	Add GitHub Actions workflow to run pytest

4. `requirements.txt` และ `requirements-dev.txt`

4 ระบุ library และเวอร์ชันที่ใช้

ปัญหาเดิม

ไม่มีไฟล์บอกว่าโปรเจกต์ต้องใช้ library อะไร ถ้าย้ายเครื่องหรือมีคนอื่นมาใช้ ต้องลองรันแล้วติดตั้งทีละตัวตาม error

แก้อย่างไร

เขียนเฉพาะ library ที่โค้ดเรา import ตรงๆ แบ่งเป็นกลุ่มพร้อม comment และ**ล็อกเวอร์ชันด้วย** `==`:

```
# Database
SQLAlchemy==2.0.52
alembic==1.19.1
psycopg2-binary==2.9.12
# Config
python-dotenv==1.2.3
# Reports
Jinja2==3.1.6
weasyprint==70.0
# AI remediation
google-genai==2.23.0
# API
fastapi==0.141.1
# Scanners (Trivy and ZAP are not Python packages - install separately)
semgrep==1.175.0
```

แยก library ที่ใช้เฉพาะตอนพัฒนาไว้ใน `requirements-dev.txt`:

```
-r requirements.txt

# Testing
pytest==9.1.1
```

ทำไมไม่ใช้ `pip freeze` ทั้งหมด

`pip freeze` จะได้ library ซ้อนมาเป็นร้อยตัว ซึ่งอ่านไม่ออกว่าตัวไหนเราใช้จริง และบางตัวผูกกับ Windows

ทำไมต้องล็อกเวอร์ชัน

ถ้า Semgrep ออกเวอร์ชันใหม่แล้วเปลี่ยนรูปแบบ JSON `normalizer.py` อาจอ่านผลผิดโดยไม่มี error
การล็อกเวอร์ชันทำให้ทุกเครื่องได้ผลเหมือนกัน

ทำไมแยกเป็น 2 ไฟล์

เครื่องที่ใช้งานจริงไม่จำเป็นต้องมี pytest บรรทัด `-r requirements.txt` แปลว่า "ติดตั้งทุกอย่างในไฟล์นั้นด้วย"

ไฟล์: `requirements.txt` (ใหม่), `requirements-dev.txt` (ใหม่)

> **บทเรียน:** comment ภาษาไทยใน `requirements.txt` แสดงเป็นตัวอ่านไม่ออกใน PowerShell
> เลยเปลี่ยนเป็นภาษาอังกฤษ ไฟล์ตั้งค่าแบบนี้ควรใช้ภาษาอังกฤษล้วน

5. pytest: ทดสอบอัตโนมัติ

5 ย้ายสคริปต์ทดลองออกจาก `tests/`

ปัญหาเดิม

โฟลเดอร์ `tests/` มีไฟล์ชื่อ `test_*.py` 9 ไฟล์ แต่จริงๆ เป็นสคริปต์ลองเรียกเครื่องมือ ไม่มี `assert` บางตัวเรียก Docker หรือ Gemini จริง ถ้าให้ pytest รันจะช้า เสียโควตา API และพังถ้าไม่มี internet

แก้อย่างไร

`git mv tests scripts` ย้ายทั้งโฟลเดอร์ไปเป็น `scripts/` แล้วสร้าง `tests/` ใหม่สำหรับ test จริงเท่านั้น (ใช้ `git mv` แทนการลากไฟล์ git จะรู้ว่าเป็นการย้าย ไม่ใช่ลบแล้วสร้างใหม่ ประวัติไฟล์ยังอยู่)

6 ตั้งค่า pytest

```
[pytest]
testpaths = tests
pythonpath = .
```

แต่ละบรรทัดทำอะไร

`testpaths = tests` ให้หา test เฉพาะใน `tests/` ไม่เข้าไปใน `scripts/` หรือ `CheatSheetSeries/`

`pythonpath = .` ให้ไฟล์ test `import normalizer` จากโฟลเดอร์หลักได้

ไฟล์: `pytest.ini` (ใหม่)

7 เขียน test 17 ตัว 17 passed

เลือก test ส่วนที่**ถ้าผิดแล้วรายงานจะผิดแบบมองไม่เห็น** และเป็นการคำนวณล้วนๆ ไม่ต้องต่อฐานข้อมูลหรือ internet

test	ป้องกันอะไร
`tests/test_risk_scoring.py` (7 ตัว)	
ERROR→high, WARNING→medium, critical→critical	แปลง severity ของแต่ละเครื่องมือให้เป็นมาตรฐานเดียว
None, "" หรือค่าแปลกๆ → info	ข้อมูลไม่ครบแล้วโปรแกรมไม่ crash
critical + CVSS 9.5 = 9.25, high + ไม่มี CVSS = 7.0	สูตร risk score ที่ใช้เรียงลำดับในรายงาน
`tests/test_normalizer.py` (10 ตัว)	
Semgrep: โค้ดเลื่อนบรรทัดแล้ว fingerprint ไม่เปลี่ยน	เพิ่มบรรทัดข้างบนแล้วช่องโหว่เดิมกลายเป็นตัวใหม่
Semgrep: 2 จุดในไฟล์เดียวกันได้ fingerprint ต่างกัน	bug เดิมที่รวมหลายจุดเป็นจุดเดียว (เคยนับได้ 22 จากจริง 35)
Semgrep: "requires login" ใช้เลขบรรทัดแทน	ทุกจุดได้ fingerprint ซ้ำกันเพราะโค้ดเป็นข้อความเดียวกัน
Semgrep: path เป็น relative, ดึง CWE-95 ได้	ย้ายโฟลเดอร์แล้ว fingerprint เปลี่ยน, จับคู่ OWASP ไม่ได้
Trivy: คนละเวอร์ชันเป็นคนละ finding, อ่าน CVSS 9.8	อัปเกรด library แล้ว finding เดิมไม่ถูกปิด, risk score ผิด

ZAP: ไม่สนใจ `?id=1` กับ `?id=2`	หน้าเดียวกันแตกเป็นหลายร้อย finding
ZAP: คนละ path เป็นคนละ finding, ตัด HTML ออกจากคำอธิบาย	รวมกันเกินไป, รายงานมีแท็ก `<p>` ปน

ไฟล์: `tests/test_risk_scoring.py`, `tests/test_normalizer.py` (ใหม่ทั้งคู่)

> **กฎของ pytest ที่ต้องจำ:** ชื่อไฟล์ต้องขึ้นต้นด้วย `test_` (ไม่ใช่ `tests_`) และชื่อฟังก์ชันต้องขึ้นต้นด้วย `test_` ถ้าไม่ตรง pytest จะหาไม่เจอแล้วขึ้น `collected 0 items`

6. กลุ่ม A: เก็บกวาดระบบ

8 ลบไฟล์เก่าที่ไม่ใช้แล้ว

ลบอะไร

`templates/output_report.html`, `templates/security_dashboard.html` (template รายงานแบบเก่า), `security_report.pdf` ที่โฟลเดอร์หลัก และ `zap_results.json` ที่โค้ดเก่าเขียนลงไปในโฟลเดอร์ source ของ NodeGoat

ทำไม

ตั้งแต่ขั้นที่ 2 รายงานไปอยู่ใน `reports/<โปรเจกต์>_<เวลา>/` ไฟล์เก่าเลยไม่มีใครใช้
ถ้าปล่อยไว้จะสับสนว่าตัวไหนคือรายงานล่าสุด

วิธี

ไฟล์ที่อยู่ใน git ใช้ `git rm` ไฟล์ที่ไม่อยู่ใน git ใช้ `Remove-Item` ถ้าไม่แน่ใจ เช็กด้วย `git ls-files <ไฟล์>` ก่อน แล้วรัน `pytest` และ `generate_report.py` เพื่อยืนยันว่าไม่ได้ลบไฟล์ที่ระบบยังใช้

9 ลบ finding ที่เป็น `fixed` ปลอม 358 รายการ ข้อมูลผิด

ปัญหา

ตอนแก้สูตร fingerprint ในขั้นที่ 1–2 ช่องโหว่เดิมได้ fingerprint ใหม่ทั้งหมด ระบบเลยเข้าใจว่าตัวเก่า "หายไปแล้ว" แล้วเปลี่ยนเป็น `fixed` และตัวใหม่เป็น `open` ทั้งที่ยังไม่มีใครแก้อะไรเลย

ทำไมต้องลบ

รายงานไม่ได้รับผลกระทบเพราะแสดงเฉพาะ `open` แต่ถ้าวันหลังทำกราฟ "แก้ไปแล้วกี่ตัว" ตัวเลขจะผิดไป 358

วิธีเลือกแถวที่จะลบ

ScanJob 29 เป็นรอบแรกที่ใช้ fingerprint ใหม่ แถวที่เป็น `fixed` และ `last_seen` ก่อนรอบนั้น คือตัวที่ใช้ fingerprint เก่าทั้งหมด

วิธีลบอย่างปลอดภัย

1) นับด้วยเงื่อนไขเดียวกันก่อน ได้ 358 ตรงกับที่คาด 2) ลบใน transaction ที่มีตัวเช็ก ถ้าจำนวนไม่ใช่ 358 พอดีจะ `ROLLBACK` เอง 3) ได้ `deleted 358 rows` แล้วค่อย `COMMIT` (ถ้าจำนวนไม่ตรงให้ใช้ `ROLLBACK;` แทน)

```
SELECT count(*) FROM findings
WHERE status = 'fixed'
  AND last_seen < (SELECT created_at FROM scanjob WHERE id = 29);

BEGIN;
DELETE FROM findings
WHERE status = 'fixed'
  AND last_seen < (SELECT created_at FROM scanjob WHERE id = 29);
COMMIT;
```

tool	ก่อน (open / fixed)	หลัง (open / fixed)
semgrep	35 / 22	35 / 0
trivy-container	119 / 105	119 / 0
trivy-dep	91 / 76	91 / 0

zap	91 / 155	91 / 0

ตาราง: `findings` (ข้อมูลอย่างเดียว ไม่ได้แก้โค้ด)

10 Backup ฐานข้อมูลด้วย `pg_dump` กันข้อมูลหาย

ปัญหาเดิม

git สำรองแค่โค้ด ข้อมูลสแกนทั้งหมดอยู่ใน Docker volume ที่เดียว ถ้าเผลอลบ volume หรือ container ข้อมูลหายถาวร

แก้อย่างไร

```
docker exec secpipeline-postgres pg_dump -U secpipeline -d secpipeline -Fc -f /tmp/
secpipeline.dump
docker cp secpipeline-postgres:/tmp/secpipeline.dump backups\secpipeline_20261008.dump
```

`-Fc` คือ format แบบบีบอัด ซึ่งตอนกู้เลือกเฉพาะบางตารางได้ ได้ไฟล์ขนาด 129 KB และตรวจด้วย `pg_restore -l` แล้วว่ามีครบทั้ง 9 ตาราง พร้อม constraint

ทำไมไม่ใช้ `>` เขียนไฟล์

PowerShell 5.1 แปลงข้อมูลที่ผ่าน `>` เป็น UTF-16 ไฟล์ backup จะเสียแบบมองไม่เห็น `docker cp` คัดลอกไฟล์ตรงๆ จึงปลอดภัยกว่า

วิธีกู้คืน (ระวัง: แทนที่ข้อมูลปัจจุบันทั้งหมด)

```
docker cp backups\secpipeline_20261008.dump secpipeline-postgres:/tmp/restore.dump
docker exec secpipeline-postgres pg_restore -U secpipeline -d secpipeline --clean --if-
exists /tmp/restore.dump
```

ไฟล์: `backups/` (ใหม่ อยู่ใน `.gitignore`)

11 `verify_findings.py` ใช้ได้กับทุกโปรเจกต์

ปัญหาเดิม

- path `C:\Allprojects\Targets\NodeGoat\package-lock.json` เขียนตายตัว
- ดึง finding ของทุกโปรเจกต์มาปนกัน และดึงตัวที่ `fixed` แล้วมาด้วย
- โค้ดรันทันทีตอน import ไม่มี `if __name__ == "__main__"`

แก้อย่างไร

- ใช้ `argparse` รับ `--project` (บังคับ) และ `--lockfile` (ไม่บังคับ)
- ถ้าไม่ใส่ `--lockfile` จะใช้ `package-lock.json` ในโฟลเดอร์ repo ที่ `run_pipeline.py` บันทึกไว้ใน `project.repo_url`
- กรองเฉพาะ `project_id` นั้น และเฉพาะ `status = 'open'`
- ถ้าไม่เจอโปรเจกต์หรือไฟล์ แสดงข้อความภาษาไทยบรรทัดเดียวแทน traceback

ผล

`python verify_findings.py --project NodeGoat` → เจอ 804 package, finding dependency 91 ตัว **ตรงกันทั้ง 91** ไม่ตรง 0 หาไม่เจอ 0 แปลว่าเวอร์ชันที่ Trivy รายงานถูกต้อง

ไฟล์: `verify_findings.py`

12 เขียน `README.md`

มีอะไรบ้าง

ระบบทำอะไร · สิ่งที่ต้องติดตั้งก่อน · วิธีติดตั้งและตั้งค่า `.env` · วิธีใช้ `run_pipeline.py` และ `generate_report.py` · วิธีรัน test · วิธี backup · ตารางแก้ปัญหาที่เจอบ่อย · โครงสร้างไฟล์

ทำไม

README คือหน้าแรกที่ GitHub แสดง คนที่ได้ประโยชน์มากที่สุดคือตัวเราเองในอีกหลายเดือน ตอนที่ลืมไปแล้วว่าต้องเปิด Docker ก่อน หรือต้องตั้งค่า Norton ยังไง

คำสั่งที่เปลี่ยนไป

ใน README ใช้ `docker compose up -d` แทน `docker start ...` เพราะอ่านค่าจาก `docker-compose.yml` เปิดทั้ง Postgres และ Redis และถ้ายังไม่มี container ก็สร้างให้

ไฟล์: `README.md` (ใหม่)

7. กลุ่ม B: GitHub Actions (CI)

13 รัน pytest อัตโนมัติทุกครั้งที่ push ☑ 17 passed บน GitHub

ปัญหาเดิม

test มีแล้ว แต่ต้องจำรันเองก่อน push ถ้าลืม โค้ดที่พังก็ขึ้น GitHub ไปได้

แก้อย่างไร

สร้าง `.github/workflows/tests.yml` บอก GitHub ว่าทุกครั้งที่ push เข้า `main` ให้เปิดเครื่อง Linux ติดตั้ง Python 3.14 และ library แล้วรัน `pytest -v`

```
name: Tests
on:
  push:
    branches: [main]
  pull_request:
jobs:
  pytest:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.14"
          cache: pip
      - run: pip install -r requirements-dev.txt
      - run: pytest -v
```

ทำไม test ผ่านบน GitHub ทั้งที่ไม่มี Postgres

test ทั้ง 17 ตัวเช็กแค่ `normalizer.py` และ `risk_scoring.py` ซึ่งเป็นการคำนวณล้วนๆ ไม่ต่อฐานข้อมูล นี่คือเหตุผลที่ควรแยก logic ที่เป็นการคำนวณออกจากส่วนที่ต่อฐานข้อมูล

ได้อะไร

ทุก commit บน GitHub มี ☑ หรือ ✗ ถ้าวันหลังแก้ fingerprint แล้วพังโดยไม่รู้ตัว จะเห็นทันที และใส่ใน portfolio ได้ว่าโปรเจกต์มี CI

ไฟล์: `.github/workflows/tests.yml` (ใหม่)

> **ถ้าวันหลังเห็น ❌:** เข้าแท็บ Actions แล้วกด run นั้น จากนั้นกดกล่อง **pytest** และขั้นที่มีกากบาท แล้วอ่าน error บรรทัดล่างๆ ส่วนใหญ่จะเป็นขั้น Install dependencies (library ติดตั้งไม่ได้) หรือ Run tests (โค้ดพังจริง)

8. ปัญหาที่เจอระหว่างทาง และบทเรียน

อาการ	สาเหตุ	วิธีแก้ / บทเรียน
`Connection refused` port 5432	Docker Desktop ปิดอยู่ Postgres เลยไม่ทำงาน	เปิด Docker Desktop แล้ว `docker compose up -d` ก่อนเริ่มงานทุกครั้ง อ่าน error จาก**บรรทัดล่างสุด**ก่อนเสมอ
คำสั่งติดกันเป็นบรรทัดเดียว (`...htmlgit add`)	copy หลายกล่องพร้อมกัน ตัวชี้นบรรทัดใหม่หายไป	copy ทีละกล่องด้วยปุ่มที่มุมกล่อง แล้วกด Enter ทีละคำสั่ง
psql prompt เป็น `secpipeline-#`	psql รอ `;` และสะสมข้อความไว้	พิมพ์ `\r` เพื่อล้าง ทุกคำสั่ง SQL ต้องจบด้วย `;`
`relation "scan_jobs" does not exist`	เดาชื่อตารางผิด ชื่อจริงคือ `scanjob`	ใช้ `\dt` ดูชื่อตารางก่อนเขียน SQL เสมอ อย่าเดา
การย้าย tests → scripts ติดเข้าไปใน commit เรื่อง CWE	`git commit` บันทึก**ทุกอย่าง**ใน Changes to be committed ไม่ใช่แค่ไฟล์ที่เพิ่ง `git add`	รัน `git status` ก่อน commit ทุกครั้ง ถ้ามีของที่ไม่เกี่ยวให้ `git restore --staged <ไฟล์>` ออกก่อน
ลบไฟล์แล้วยังอยู่ใน Changes not staged	ลบด้วย `Remove-Item` หรือลบเองแทน `git rm`	รัน `git rm` ซ้ำกับไฟล์ที่หายไปแล้วได้
`nothing to commit` หลัง `git add README.md`	README ถูก commit ไปแล้วก่อนหน้า	ไม่ใช่ error ดูด้วย `git log --oneline`
GitHub Actions ไม่ขึ้น	สร้างไฟล์แล้วแต่ยังไม่ได้ add, commit, push	GitHub เห็นเฉพาะสิ่งที่ผ่านครบ `add → commit → push`
`pytest.ini:1: unexpected line: 'ini'`	copy ชื่อภาษาของกล่องโค้ดติดมาด้วย	บรรทัดแรกต้องเป็น `[pytest]`
`collected 0 items`	ตั้งชื่อไฟล์ `tests_risk_scoring.py`	ต้องขึ้นต้นด้วย `test_` และใช้ `Rename-Item` เพราะไฟล์ยังไม่อยู่ใน git (`git mv` ใช้ไม่ได้)
แก้ไฟล์ใน Notepad แล้วไม่ถูกบันทึก	Notepad บน Windows 11 เปิดแท็บเก่าค้างไว้	ใช้ `code <ไฟล์>` (VS Code) และเช็ก LastWriteTime ด้วย `dir`
`normalizer.py` พังหลังทดลองแก้	แก้โค้ดค้างไว้ครึ่งทาง	`git restore normalizer.py` กลับไปใช้ commit ล่าสุด (นี่คือประโยชน์ของ git)
ภาษาไทยเพี้ยนใน `Get-Content`	PowerShell 5.1 ไม่อ่านไฟล์เป็น UTF-8	ดูไฟล์ใน VS Code แทน และใส่ `-Encoding utf8` ถ้าต้องใช้ `Set-Content`

> **นิสัย 4 ข้อที่กันพังได้มากที่สุด** ทุกครั้งที่แก้อะไร: **1)** `git status` ดูว่าแก้อะไรไป **2)** `pytest` ต้องผ่านหมด **3)** รันจริงหนึ่งรอบแล้วดูผล **4)** `git commit` แล้ว `git push` และก่อนแก้หรือลบข้อมูลในฐานข้อมูล ให้ `pg_dump` ก่อนทุกครั้ง

9. สรุปคำสั่งที่ได้เรียน

git

คำสั่ง	ใช้เมื่อ
`git status`	ดูว่าไฟล์ไหนเปลี่ยน และไฟล์ไหนเตรียม commit แล้ว (ใช้บ่อยที่สุด)
`git add <ไฟล์>`	เตรียมไฟล์ไว้ commit
`git commit -m "ข้อความ"`	บันทึกจุด save
`git push`	ส่งขึ้น GitHub
`git log --oneline -5`	ดู commit ล่าสุด 5 ตัว
`git mv <เก่า> <ใหม่>`	ย้ายหรือเปลี่ยนชื่อไฟล์ที่อยู่ใน git
`git rm <ไฟล์>`	ลบไฟล์ที่อยู่ใน git
`git ls-files <ไฟล์>`	เช็กว่าไฟล์อยู่ใน git ไหม
`git restore <ไฟล์>`	ทิ้งการแก้ไข กลับไปเหมือน commit ล่าสุด
`git restore --staged <ไฟล์>`	เอาออกจากรายการเตรียม commit (ไฟล์ไม่เปลี่ยน)

Docker และ Postgres

คำสั่ง	ใช้เมื่อ
`docker compose up -d`	เปิด Postgres และ Redis (ทำทุกครั้งที่เปิดเครื่อง)
`docker ps -a`	ดูว่า container ไหนทำงานหรือหยุดอยู่
`docker exec -it secpipeline-postgres psql -U secpipeline -d secpipeline`	เข้า psql เพื่อพิมพ์ SQL
`\dt` / `\r` / `\q`	ดูชื่อตาราง / ล้างคำสั่งที่ค้าง / ออก (ใช้ใน psql)
`BEGIN;` … `COMMIT;` หรือ `ROLLBACK;`	แก้หรือลบข้อมูลแบบยกเลิกได้
`pg_dump -Fc` + `docker cp`	backup ฐานข้อมูล
`pg_restore -l`	เช็กว่าไฟล์ backup เปิดได้

Python และโปรเจกต์

คำสั่ง	ใช้เมื่อ
`pip install -r requirements-dev.txt`	ติดตั้ง library ทั้งหมด
`pytest -v`	รัน test ทั้งหมดและแสดงชื่อแต่ละตัว
`python run_pipeline.py --project X --repo ... --image ... --url ...`	สแกนแล้วสร้างรายงาน
`python generate_report.py --project X`	สร้างรายงานใหม่โดยไม่ต้องสแกน
`python verify_findings.py --project X`	เทียบผล Trivy กับ `package-lock.json`
`code <ไฟล์>`	เปิดไฟล์ใน VS Code

10. สถานะงาน และสิ่งที่ยังเหลือ

เทียบกับ "งานที่ยังเหลือ" ในรายงานขั้นที่ 1–2

งานที่ค้างไว้	สถานะ
finding `fixed` ปลอม 358 รายการ	**เสร็จ** ลบแล้ว (ข้อ 9)
`verify_findings.py` path ตายตัว	**เสร็จ** (ข้อ 11)
ไฟล์เก่าที่ไม่ใช้แล้ว	**เสร็จ** (ข้อ 8) ยกเว้นตาราง backup 2 ตาราง
`requirements.txt`	**เสร็จ** (ข้อ 4)
test ด้วย pytest	**เสร็จ** 17 test (ข้อ 5–7) ยังไม่มี test ของ `normalize_cwe_id`
commit ขึ้น git	**เสร็จ** (ข้อ 2–3) พร้อม CI (ข้อ 13)
Gemini อยู่ใน pipeline	**ยังไม่ทำ** กลุ่ม C1
API และงานเบื้องหลัง	**ยังไม่ทำ** กลุ่ม C3
EPSS ใน risk score	**ยังไม่ทำ** กลุ่ม C2
สแกนหลาย image ในโปรเจกต์เดียว	**ยังเป็นข้อจำกัด**

เรื่องเล็กที่เจอระหว่างทาง (ยังไม่ได้แก้)

เรื่อง	รายละเอียด
ScanJob 27 ค้าง `running`	ไม่มีเวลาสร้าง น่าจะเป็นรอบที่หยุดกลางทางตั้งแต่ก่อนแก้โค้ด ลบได้
ตาราง `cwe_owasp_mapping_backup` และ `_backup_20261001`	ลบได้เมื่อมั่นใจว่า `cwe_owasp_mapping` ถูกต้อง (ตอนนี้มีอยู่ในไฟล์ `pg_dump` แล้วด้วย)
`.env.example`	ไฟล์ตัวอย่างที่มีแค่ชื่อตัวแปร ไม่มีค่า ใช้ commit ขึ้น git ได้ ช่วยให้รู้ว่าต้องตั้งค่าอะไรบ้าง
backup อัตโนมัติ	ตอนนี้ต้อง `pg_dump` เอง ทำเป็นสคริปต์หรือตั้งเวลาได้
test เพิ่ม	`normalize_cwe_id`, `extract_location_detail` และ `save_findings` (ส่วนหลังต้องใช้ฐานข้อมูลทดสอบ)

ขั้นต่อไป (กลุ่ม C)

ข้อ	งาน	ได้อะไร
C1	`--with-remediation`	สแกนเสร็จแล้วเรียก Gemini เขียนคำแนะนำให้ CWE ใหม่ในคำสั่งเดียว
C2	EPSS ใน risk score	ดึงคะแนน "โอกาสถูกโจมตีจริง" จาก FIRST.org (คอลัมน์ `epss_score` มีรอไว้แล้ว) ช่องโหว่ที่มีคนโจมตีจริงจะขึ้นมาอยู่บนสุด
C3	FastAPI และ Redis worker	สั่งสแกน ดูสถานะ และดาวน์โหลดรายงานผ่าน API เป็นพื้นฐานของหน้าเว็บในอนาคต

ไฟล์ที่เปลี่ยนในรอบนี้

ไฟล์	สถานะ
`requirements.txt`, `requirements-dev.txt`	**ใหม่** รายชื่อ library และเวอร์ชัน
`pytest.ini`	**ใหม่** ตั้งค่า pytest
`tests/test_risk_scoring.py`, `tests/test_normalizer.py`	**ใหม่** 17 test
`scripts/`	**ย้าย** จาก `tests/` เดิม
`seed_cwe_mapping.py`, `mappings/cwe_owasp_top10.json`	**ขึ้น git ครั้งแรก**
`verify_findings.py`	**แก้** argparse, กรองโปรเจกต์และ open
`README.md`	**ใหม่** คู่มือ
`.github/workflows/tests.yml`	**ใหม่** CI
`.gitignore`	**แก้** เพิ่ม `CheatSheetSeries/`, `backups/`
`templates/output_report.html`, `templates/security_dashboard.html`	**ลบ**
`.env`, `ca-bundle.pem`	**คุณทำเอง** certificate ของ Norton (ไม่อยู่ใน git)
ฐานข้อมูล: ตาราง `findings`	**ลบข้อมูล** 358 แถว